Report

1. Aim

The aim of this experiment is to evaluate the performance of a C-based HTTP server under different levels of concurrent client load and compare three request-processing designs:

1. Sequential server (s) — handles requests sequentially.
2. Thread-per-request server (m) — creates a separate thread to handle each request.
3. Thread-pool server (t) — uses a fixed number of worker threads to handle incoming requests.

The server is evaluated using three functions with different computational characteristics:

- /square?num=3 — a lightweight arithmetic operation.
- /helloworld?str=Ashwin — a lightweight string-processing operation.
- /arithmetic/prime?num=1000000007 — a CPU-intensive prime-number computation.

The experiment investigates how throughput and mean response time change with increasing concurrency, how thread-pool size affects performance, and whether the benefits of multithreading depend on the workload.

2. Experimental setup and execution

2.1 Machine and CPU assignment

The experiments were run on Kali Linux(VM, already had it setup) using the `cflask` server and `wrk` HTTP benchmarking tool.

Parameter	Configuration
Operating system	Kali Linux VM
CPU model	AMD Ryzen 5 5600X
Total logical CPUs	8
Server CPU affinity	CPUs `0-2`
Load-generator CPU affinity	CPUs `4-7`
Load-generator threads	1
Test duration	30 seconds per test
Concurrent connections	1, 2, 4, 8, 16, 32, 64, 128

2.2 Server configurations and commands

The following are the server-start commands corresponding to the six configurations. They assume the server executable is `./bin/cflask` and the CPU assignment is `0-2`.

1. **Sequential server:** taskset -c 0-2 ./bin/cflask s 8080
2. **Thread-per-request server:** taskset -c 0-2 ./bin/cflask m 8080
3. **Thread pool with 1 worker:** taskset -c 0-2 ./bin/cflask t 8080 1
4. **Thread pool with 2 workers:** taskset -c 0-2 ./bin/cflask t 8080 2
5. **Thread pool with 4 workers:** taskset -c 0-2 ./bin/cflask t 8080 4
6. **Thread pool with 8 workers:** taskset -c 0-2 ./bin/cflask t 8080 8

Only one server configuration ran on port 8080 at a time.

2.3 Load-generator commands

The benchmark script used `wrk` with one generator thread, varying the number of connections and running each test for 30 seconds. The exact command format was:

taskset -c 4-7 ./wrk -t 1 -c 1 -d 30s --latency "http://127.0.0.1:8080/square?num=3"

The same command structure was used for the other connection counts and URLs. For example:

taskset -c 4-7 ./wrk -t 1 -c 128 -d 30s —latency "http://127.0.0.1:8080/arithmetic/prime?num=1000000007"

The three tested URLs were:

Function	URL
Square	http://127.0.0.1:8080/square?num=3
Hello World	http://127.0.0.1:8080/helloworld?str=Ashwin
Prime	http://127.0.0.1:8080/arithmetic/prime?num=1000000007

2.4 Independent parameters and metrics

Independent parameter: The main independent parameter was the number of concurrent client connections, controlled using wrk -c. The server-processing design was also varied across six configurations: sequential, thread-per-request, and thread pools containing 1, 2, 4, or 8 workers.

Measured metrics:

- Throughput (requests/sec): Number of HTTP requests completed per second, as reported by wrk.
- Mean response time (ms): Average request latency reported by wrk, converted to milliseconds when necessary.

For every function, two plots were produced: throughput against concurrent connections and mean response time against concurrent connections. Each plot compares all server configurations.

2.5 Hypotheses

The following are the hypotheses are made before running the experiment,

- H1 — Lightweight functions: The sequential server may perform competitively for simple operations because it avoids thread creation, synchronization, and scheduling overhead.
- H2 — Thread-per-request: This design may perform worse at high concurrency because creating and managing many threads introduces overhead.
- H3 — Thread pools: A thread pool may provide better scalability than thread-per-request because workers are reused. Increasing the pool size may improve throughput until CPU or synchronization overhead becomes a bottleneck.
- H4 — CPU-intensive function: The prime-number computation may benefit more from multiple worker threads because requests require more CPU work, allowing useful parallel execution.
- H5 — Response time: Mean response time is expected to increase when the server becomes saturated, even if throughput has stopped increasing.

3. Experiment 1: Square function

3.1 Aim

To determine how the six server configurations behave under increasing concurrency for a lightweight arithmetic operation, and whether multithreading provides a measurable advantage when each request requires little computation.

3.2 Observations

The sequential server achieved the highest measured throughput for this function.

Server configuration	Peak throughput (req/s)	Connection count at peak
Sequential	9,020.48	16
Thread-per-request	2,941.58	4
Thread pool — 1 worker	6,556.57	128
Thread pool — 2 workers	5,823.74	128
Thread pool — 4 workers	5,318.07	128
Thread pool — 8 workers	4,816.43	8

The sequential server's throughput increased rapidly as concurrency rose from 1 to 16 connections, reaching approximately 9,020 requests/sec. Beyond 16 connections, throughput largely flattened, remaining around 8,700–8,850 requests/sec.

The thread-per-request server peaked at approximately 2,942 requests/sec with 4 connections, then generally declined as concurrency increased. Its throughput at 128 connections was 2,302 requests/sec.

The one-worker thread pool increased steadily from approximately 1,754 requests/sec at one connection to 6,557 requests/sec at 128 connections. Pools with 2 and 4 workers also improved at higher connection counts, but neither surpassed the sequential server. The eight-worker pool reached its peak at 8 connections and then fluctuated at a lower throughput.

Mean response time generally increased at high concurrency, although there were irregularities and large spikes in some measurements. For example, the sequential server's reported mean latency was 1.67 ms at 16 connections and 9.95 ms at 32 connections. The thread-per-request server reported 35.44 ms at 64 connections.

3.3 Explanation and inferences

The results support the hypothesis that a sequential server can be highly efficient for lightweight request handlers. Since the square operation requires little CPU work, the additional overhead of creating threads or coordinating a worker pool may outweigh the benefit of parallel execution.

The thread-per-request implementation appears particularly inefficient for this workload. Creating a new thread for each incoming request can consume time and system resources, especially as concurrency increases.

The pool results also show that increasing the number of worker threads does not necessarily increase throughput. For a small arithmetic operation, a pool may spend a significant fraction of its time on scheduling, coordination, and request dispatch rather than computation.

The hypothesis is therefore supported for this workload under the measured setup, but the results do not establish that sequential servers are always faster. The conclusion applies to this specific endpoint, implementation, and test environment.

4. Experiment 2: Hello World function

4.1 Aim

To compare the server designs for a lightweight string-processing operation and determine whether its performance follows the same pattern as the square function.

4.2 Observations

The sequential server again achieved the highest peak throughput.

Server configuration	Peak throughput (req/s)	Connection count at peak
Sequential	9,121.98	16
Thread-per-request	2,924.03	4
Thread pool — 1 worker	6,880.98	128
Thread pool — 2 workers	5,882.74	128
Thread pool — 4 workers	5,311.96	128
Thread pool — 8 workers	4,908.18	8

The sequential server reached approximately 9,122 requests/sec at 16 connections. Throughput then remained close to 9,000 requests/sec at 32 and 64 connections before falling slightly at 128 connections.

The thread-per-request server peaked at approximately 2,924 requests/sec at 4 connections and stayed near 2,600–2,700 requests/sec at higher concurrency.

The one-worker pool reached approximately 6,881 requests/sec at 128 connections. The two-worker and four-worker pools reached approximately 5,883 and 5,312 requests/sec respectively at 128 connections. The eight-worker pool peaked at approximately 4,908 requests/sec at 8 connections and did not maintain that throughput as concurrency increased.

Mean response time generally rose with concurrency, particularly for thread-per-request. Its mean latency increased from approximately 0.47 ms at one connection to 22.52 ms at 128 connections. The sequential server also exhibited a large latency increase at some higher connection counts, although its throughput remained relatively stable.

4.3 Explanation and inferences

The Hello World function exhibits a pattern similar to the square function. Both perform relatively little computation per request, so the sequential server can process requests efficiently without incurring multithreading overhead.

The fact that the one-worker pool performs better than the thread-per-request configuration at high concurrency is consistent with the benefit of reusing worker threads rather than creating a new thread for each request.

However, the multi-worker pools did not consistently outperform the one-worker pool. This suggests that additional workers introduce costs that are not compensated for by the small amount of work performed per request. Other implementation details, such as queue locking, socket handling, and scheduling, may also affect performance.

Thus, the results support the lightweight-workload hypothesis: multithreading is not automatically beneficial when request processing is inexpensive.

5. Experiment 3: Prime-number function

5.1 Aim

To evaluate how the server configurations behave when each request performs a CPU-intensive prime-number computation, and determine whether multiple worker threads improve throughput under heavier computational demand.

5.2 Observations

The results differ significantly from those of the lightweight functions.

Server configuration	Peak throughput (req/s)	Connection count at peak
Sequential	3,820.35	16
Thread-per-request	2,690.62	4
Thread pool — 1 worker	4,311.42	8
Thread pool — 2 workers	4,950.03	8
Thread pool — 4 workers	7,099.39	128
Thread pool — 8 workers	7,423.96	128

The sequential server reached approximately 3,820 requests/sec at 16 connections. Its throughput then dropped to approximately 1,977 requests/sec at 32 connections and 1,293 requests/sec at 64 connections, before increasing to 2,798 requests/sec at 128 connections.

The thread-per-request server peaked at approximately 2,691 requests/sec at 4 connections and did not show sustained throughput growth at higher concurrency.

The one-worker pool reached approximately 4,311 requests/sec at 8 connections before its throughput declined at higher concurrency. The two-worker pool peaked at approximately 4,950 requests/sec at 8 connections and maintained throughput above 4,300 requests/sec through 128 connections.

The four-worker pool showed a strong increase in throughput as concurrency rose, reaching approximately 7,099 requests/sec at 128 connections. The eight-worker pool performed best overall, reaching approximately 7,424 requests/sec at 128 connections.

Mean response time generally increased with concurrency, but the pattern was irregular. For example, the four-worker pool reported approximately 1.13 ms at 32 connections and 1.66 ms at 64 connections, then 9.81 ms at 128 connections. The eight-worker pool reported approximately 1.02 ms at 32 connections, 1.89 ms at 64 connections, and 7.64 ms at 128 connections.

The sequential server's large throughput variations at higher connection counts coincide with irregular latency measurements and socket timeouts in the recorded output. Those results should therefore be interpreted cautiously.

5.3 Explanation and inferences

Unlike the first two functions, prime-number computation involves more CPU work per request. With only one execution path, the sequential server cannot process multiple CPU-intensive requests simultaneously. As the number of outstanding requests increases, requests can wait longer for their turn, and throughput may eventually stop increasing.

The thread pool allows multiple requests to be processed concurrently. The four-worker and eight-worker pools substantially outperform the sequential server at high concurrency, which supports the hypothesis that a CPU-intensive workload can benefit from parallel request handling.

The eight-worker pool's measured throughput of approximately 7,424 requests/sec at 128 connections is about 1.94 times the sequential server's peak throughput of 3,820 requests/sec. This comparison uses the best observed throughput for each configuration, not the same connection count, so it should be treated as an indication of their overall measured performance rather than a controlled same-load speedup.

The thread-per-request server does not achieve the same benefit. This is consistent with the possibility that frequent thread creation and scheduling overhead reduce the benefits of parallel execution.

The results broadly support the hypothesis that worker pools are more useful for CPU-intensive work. Nevertheless, the throughput and latency fluctuations at high concurrency suggest that the implementation, resource limits, and measurement stability also affect the results.

6. Comparison across all experiments

The three functions demonstrate that the best server design depends on the amount of work performed by each request.

6.1 Comparison of peak throughput

The analysis, plots & results provided shows:

- Lightweight functions: The sequential server achieves the highest peak throughput for both square and Hello World, at approximately 9,020 and 9,122 requests/sec respectively.
- Thread-per-request: This configuration has the lowest or nearly lowest peak throughput across the three functions. Its thread-management overhead appears to outweigh the benefits of concurrent execution in this implementation.
- CPU-intensive function: The four-worker and eight-worker pools achieve the highest peak throughput, at approximately 7,099 and 7,424 requests/sec respectively. This is consistent with parallel execution being more valuable for computationally expensive requests.

6.2 Comparison of scalability

For the square and Hello World functions, the sequential server reaches its highest throughput around 16 concurrent connections. Increasing concurrency beyond that point produces little improvement. This is a typical saturation pattern: the server is already processing requests near its effective capacity.

The thread pools for these functions generally perform better as concurrency increases, particularly the one-worker pool, but they do not surpass the sequential server in peak throughput.

For the prime function, the four-worker and eight-worker pools continue to gain throughput at higher concurrency, reaching their best measured results at 128 connections. This indicates that the workload can benefit from multiple workers and that additional concurrency helps keep them occupied.

However, increasing concurrency does not guarantee lower latency. At high loads, requests may spend more time waiting in queues or competing for resources. The observed latency spikes illustrate this trade-off.

6.3 General conclusions

The experiments lead to the following conclusions:

1. Server performance depends on the workload. A sequential server can be highly effective for lightweight handlers, while a thread pool may be more appropriate for CPU-intensive work.

2. Thread-per-request has a cost. Creating and managing a thread for each request can limit scalability, especially when requests are small or concurrency is high.

3. More workers are not always better. Additional threads introduce scheduling, coordination, and resource costs. Their benefits depend on whether the workload provides enough useful parallel work.

4. Throughput and latency should be considered together. A server can maintain or improve throughput while individual requests experience longer delays.

5. Concurrency and parallelism are different. Increasing `wrk -c` creates more simultaneous outstanding requests, but the actual amount of parallel processing depends on the server's design and available CPU resources.

Overall, the results support the view that there is no universally best request-processing architecture. A sequential server is competitive for lightweight operations, whereas a suitably sized thread pool can provide better throughput for CPU-intensive operations.

7. Limitations

The experiments were conducted under fixed CPU-affinity settings to ensure a fair comparison across all functions and server configurations.

- **Fixed CPU allocation:** The server and load generator were pinned to separate, fixed sets of CPU cores. The server was restricted to CPUs 0-2, while wrk was restricted to CPUs 4-7. This separation reduces interference between request processing and load generation and maintains consistent CPU assignments across experiments.

- **Fixed load-generator thread count:** wrk used a single thread (-t 1) for all tests, even when the number of concurrent connections exceeded four. This kept the load-generation configuration consistent while varying the number of connections using -c. Consequently, the experiments evaluate the effect of increasing concurrency with a fixed load-generator thread count.

- **Limited server CPU allocation:** The server was restricted to three CPU cores, even when the thread pool contained four or eight worker threads. Therefore, configurations with more than three workers did not have access to additional CPU cores. Multiple workers had to share the available CPU resources, which may limit the performance gains from increasing the pool size.

Despite these limitations, the fixed CPU assignments and consistent load-generator thread count provide a common testing environment for comparing the six server configurations. However, the results should be interpreted in the context of the three-core server allocation: they compare the designs under the same CPU budget, rather than measuring the maximum performance each design could achieve with unrestricted CPU resources.

8. Final conclusion

This experiment compared sequential, thread-per-request, and thread-pool implementations of a C-based HTTP server under increasing concurrency. The lightweight square and Hello World functions achieved their highest measured throughput with the sequential server, while the CPU-intensive prime function benefited significantly from pools of four and eight workers. The thread-per-request implementation did not scale as effectively in these measurements, suggesting that thread creation and scheduling overhead can outweigh parallelism benefits. The overall results show that server architecture should be selected based on workload characteristics, and that both throughput and response time must be considered when evaluating scalability.